

Unification of Real-Digital Twins for HRC Research using Isaac Sim

Diego Quezada Colorado¹, Mahmoud Chick Zaouali², Todd Charter², Josué González-García¹, Homayoun Najjaran^{2*}

¹Tecnológico de Monterrey, Santiago de Queretaro, QRO, Mexico

²University of Victoria, Victoria, BC, Canada

*najjaran@uvic.ca

Abstract—This work presents a Digital Twin (DT) architecture for a multi-robot cell developed using NVIDIA Isaac Sim and ROS 2. The system integrates two industrial manipulators, dual grippers, and a direct-drive linear actuator within a unified communication framework that enables real-time joint-state mirroring and synchronized trajectory execution. To evaluate the fidelity of the Real-to-Sim pipeline, repeated trajectory experiments were conducted, yielding an average joint-space tracking error of 0.037 rad and an end-to-end latency of approximately 363 ms. A multi-component analysis further decomposes the influence of ROS 2 middleware transfer, simulator scheduling, and discretized joint-command execution on overall synchronization performance. The results demonstrate stable and consistent Real-to-Sim replication suitable for trajectory visualization, workspace validation, and offline task rehearsal. The proposed architecture is modular, reproducible, and hardware-agnostic, providing a foundation for future extensions involving dynamic scene reconstruction, perception-driven environment updates, immersive teleoperation, and multi-robot coordination.

Keywords-component—Digital Twin; Isaac Sim; ROS 2; Robotics; Real-Time Simulation; Industrial Automation

I. INTRODUCTION

Industry 4.0 continues to accelerate the adoption of cyber-physical systems, data-driven automation, and intelligent robotic cells. Modern industrial environments increasingly rely on simulation frameworks to enable safe testing, predictive maintenance, process optimization [1], [2], and advanced human-robot collaboration (HRC). These requirements have emphasized the need for high-fidelity virtual environments capable of accurately replicating robot behavior, environmental constraints, and system dynamics with minimal latency [3], [4].

Digital Twins (DTs) have emerged as a key enabling technology by synchronizing physical robotic systems with their

virtual counterparts in real time [3], [5]. In robotics, DTs provide a controlled environment for safe experimentation, facilitate precise trajectory verification, and scalable prototyping without disrupting production workflows [6], [7]. Despite this rapid progress, several challenges continue to limit the performance and reproducibility of DT implementations. Many rely on low-fidelity or custom simulators that do not fully capture robot kinematics, articulation constraints, or timing behavior [8]. Real-to-simulation (Real-to-Sim) synchronization remains difficult due to clock misalignment, variable communication latency, and insufficient joint-level trajectory streaming [9].

Previous research has demonstrated real-time DT mirroring and teleoperation with latencies in the tens to low hundreds of milliseconds and sub-degree joint tracking error, most commonly evaluated in single-robot configurations [10]–[13]. However, real industrial environments frequently involve robotic cell-systems, where multiple manipulators, grippers, and auxiliary actuators share computation, communication bandwidth, and control pipelines [14]. Under these conditions, concurrency introduces resource contention and timing variability that can significantly affect mirroring stability and tracking fidelity.

As a result, existing literature provides limited guidance on how Digital Twin architectures should be structured, evaluated, and optimized for multi-robot cells, particularly when deployed using modern GPU-accelerated simulators such as NVIDIA Isaac Sim. Most prior work demonstrates feasibility on isolated robots but does not systematically analyze how simulation scheduling, middleware behavior, and control-pipeline design collectively influence real-time fidelity in multi-component DT systems.

This work addresses these challenges by developing and validating a Digital Twin architecture for a multi-robot cell using NVIDIA Isaac Sim and standardized robotics interfaces. Isaac Sim is selected for its GPU-accelerated physics, photorealistic

This research was supported by Mitacs through the Mitacs Globalink program.

rendering, and native robotics workflows [15], [16], enabling integration with advanced perception and control tools across industrial and research environments. The objective is to achieve reliable Real-to-Sim joint-trajectory mirroring while maintaining low latency and joint-space deviations within acceptable industrial tolerances. The proposed architecture is modular, scalable, and hardware-agnostic, supporting extension to additional robotic platforms beyond those evaluated in this work.

The main contributions of this work are as follows:

- 1) **A reproducible Digital Twin architecture for multi-robot cells implemented in NVIDIA Isaac Sim**, integrating two industrial manipulators, dual grippers, and a direct-drive linear actuator through unified ROS 2 interfaces. The system supports real-time joint-state mirroring, synchronized trajectory execution, and modular expansion to additional robots and sensing modalities.
- 2) **A systematic Real-to-Sim synchronization analysis** based on repeated experimental trials, providing a structured evaluation of joint-space tracking fidelity and end-to-end latency within the proposed Digital Twin architecture.
- 3) **A multi-component decomposition of latency sources within a ROS 2–Isaac Sim DT pipeline**, examining contributions from middleware transfer overheads, simulation update scheduling, and joint-command discretization. This analysis offers practical insights for optimizing real-time DT performance in GPU-accelerated simulation environments.
- 4) **A modular and extensible foundation for future perception-aware and multi-robot DT research**, enabling integration of RGB-D sensing, dynamic scene reconstruction, immersive teleoperation, and closed-loop Sim-to-Real feedback. The proposed architecture emphasizes reproducibility and hardware-agnostic design suitable for deployment across different robotic platforms.

II. RELATED WORK

A. Robotic Digital Twins and Simulation Platforms

Robotic Digital Twins (DTs) are increasingly adopted in Industry 4.0 environments to support safe experimentation, predictive maintenance, and process optimization [1], [2]. They enable trajectory verification and scalable prototyping for industrial manipulators [6], [7]. Unity and Gazebo currently lead the academic landscape, being the two most frequently cited DT platforms as of late 2024 [10], with additional validation of Unity-based DTs for multi-robot simulation reported in recent studies [8].

A comparative study evaluating Digital Twins developed in both platforms reported a latency of 108.79 ± 57.65 ms and joint RMSE of $0.82 \pm 0.17^\circ$ for a Gazebo-based DT of a 6-DOF ABB IRB 1200-5/0.9 robot [10]. Using the same methodology and DT architecture, the Unity-based implementation achieved improved performance, with latency of 77.67 ± 15.74 ms and RMSE of $0.49 \pm 0.11^\circ$ [11].

Similarly, real-time teleoperation experiments reported by [12] achieved end-to-end latencies between 84 ms and 102 ms for a Unity-based IoT–VR teleoperation pipeline. Comparable DT-driven teleoperation frameworks have also been proposed using modular and plug-and-play architectures [13], [17].

Although these studies highlight the feasibility of real-time robotic Digital Twins, they focus almost entirely on single-robot configurations and do not address the scalability or reproducibility of DT architectures when extended to multi-robot cells.

B. Toward Cell-Level Digital Twins

Most existing DT evaluations focus on one manipulator at a time, either for state mirroring or isolated teleoperation pipelines [10]–[12]. However, many industrial environments rely on robotic cells with multiple manipulators operating within the same workspace and sharing compute and network resources [4], [14]. Under these conditions, concurrent message streaming, bandwidth limitations, and scheduler interactions can introduce variability that affects synchronization performance [9].

Despite the importance of such settings, cell-level Digital Twin evaluations remain under-reported. Prior literature rarely evaluates how DT performance scales in workcells that include more than one robot, even when the underlying DT architectures are nominally capable of supporting them [3], [5].

This gap motivates the development of DT systems that are structurally designed for multi-robot cells, even if early validation focuses on a single manipulator. Such architectures provide a foundation for future cell-level benchmarking once coordinated motion, shared trajectories, or dual-arm tasks are introduced.

III. METHODOLOGY

A. System Overview

We develop a modular Digital Twin (DT) architecture for a robotic cell that mirrors physical joint motion into a high-fidelity simulation environment in real time [16], [18]. The system comprises three primary components: (i) a physical UFACTORY-based robotic cell, (ii) a DT implemented in NVIDIA Isaac Sim, and (iii) a communication layer that streams joint states, replicates commands in the simulator, and logs synchronized data for evaluation. Figure 1 illustrates the overall data flow.

B. Physical and Digital Subsystems

The physical subsystem includes a UFACTORY xArm6, a UF850, two xArm grippers, and a direct-drive linear motor. An Intel RealSense D435i camera is also included to support future work involving perception and dynamic scene reconstruction. Figure 2 shows the laboratory setup. All robots interface through Ethernet to a router connected to the host workstation.

The digital subsystem is implemented in Isaac Sim as a virtual replica of the physical cell. It includes articulated

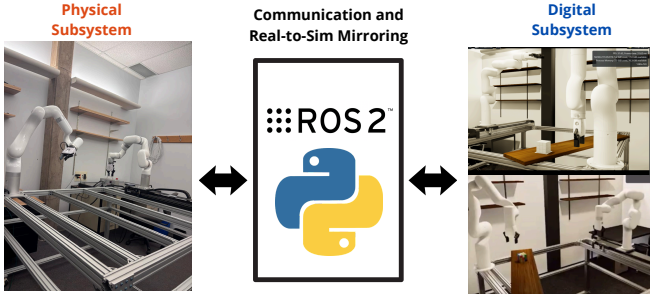


Figure. 1: System overview: ROS 2/Python bridges the UFAC-TORY robotic cell and its Isaac Sim digital twin for real-time state and command mirroring.

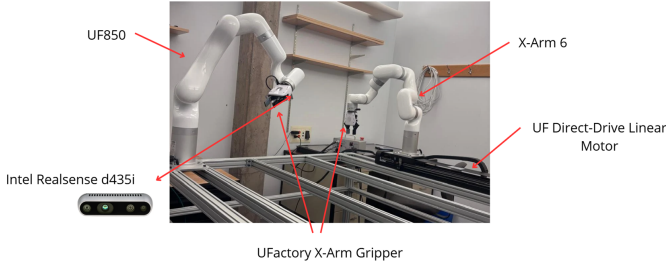


Figure. 2: Robotic Cell Physical Setup: The setup comprises 2 UFactory robotic arms, 2 parallel grippers, a linear motor, and a single RGB-D camera

USD models of both manipulators and grippers with physics-based joint dynamics and collision models. A movable prim represents the direct-drive linear actuator, with the xArm6 mounted on top. The surrounding laboratory structures are also modeled with physics enabled. Simulator parameters (e.g., articulation settings and solver configuration) are tuned to ensure stable joint-level execution suitable for Real-to-Sim mirroring experiments (Figure 3).

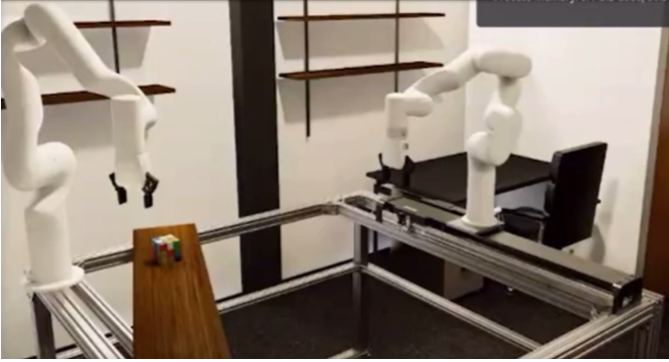


Figure. 3: Digital Twin Environment: Simulated layout in NVIDIA Isaac Sim mirroring the physical robotic cell's spatial configuration and hardware constraints

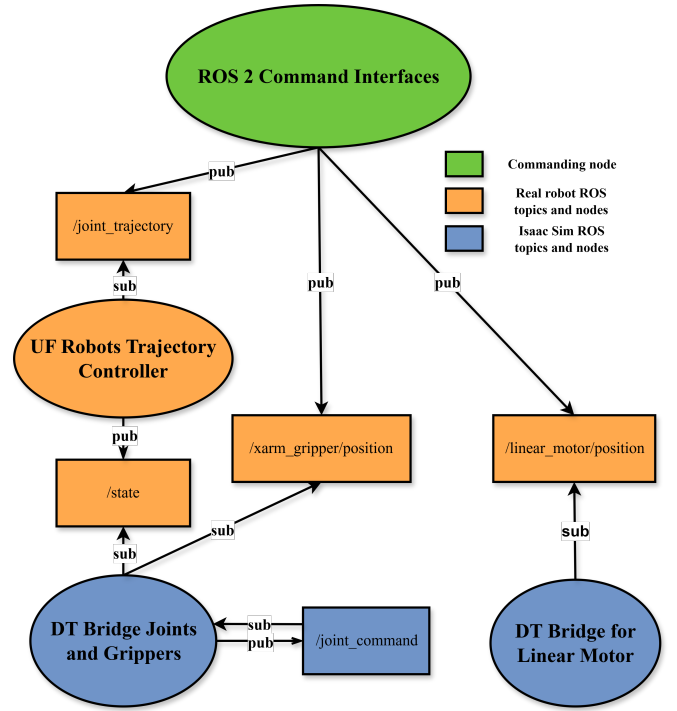


Figure. 4: Digital Twin Control Workflow: ROS 2-based communication pipeline illustrating the control flow from commanding interfaces through physical manipulators to their virtual counterparts in Isaac Sim.

C. Communication and Real-to-Sim Mirroring

Real-to-Sim mirroring is implemented using a set of ROS 2 middleware nodes together with the Isaac Sim ROS 2 bridge. Joint states from the physical system are streamed via the `/joint_states` topic at 100 Hz and consumed by a dedicated acquisition node. These measurements are forwarded to a state replication node, which maps the incoming physical joint positions to the corresponding command interfaces in the simulator. The replication node publishes joint commands through the Isaac-ROS bridge using a structured namespace scheme, ensuring isolation between topics while supporting multiple manipulators (Figure 4). This design allows both the robotic arms and the linear actuator to be addressed consistently within a single ROS 2 environment.

IV. EXPERIMENTAL SETUP

A. Compute Platform and Software Configuration

Experiments were conducted on a Linux workstation equipped with an AMD Ryzen 9 5900X (12 cores) and an NVIDIA TITAN RTX GPU, running Ubuntu 22.04. The DT environment was executed in Isaac Sim 5.0.0. Since Isaac Sim performance depends heavily on GPU capability, reproductions should follow the official Isaac Sim hardware recommendations.



Figure. 5: Experimental Home Configuration. Default joint-space pose of the xArm6 manipulator utilized as the starting state for all test cycles.

B. Initialization and Execution Procedure

To ensure consistent startup conditions across trials, two launch pipelines were used. The first initializes the physical cell by starting the robot drivers and interface nodes (arm control, grippers, and linear motor). The second launches the Isaac Sim environment, enables the simulator bridge, instantiates the articulated robot models, and activates the mirroring graph used for joint replication. After initialization, Real-to-Sim mirroring is run continuously while the planned trajectories are executed on the physical robot.

C. Trajectory Design

A joint-space trajectory was executed on the xArm6, which was the only manipulator receiving motion commands during the reported experiments. The trajectory consists of four target poses executed over 40 s and densely interpolated into over 4,000 setpoints to produce continuous motion across all six joints. The robot begins from a calibrated nominal home configuration (Figure 5). The trajectory is applied to the physical robot, while the DT receives replicated joint commands through the mirroring pipeline. The trajectory begins from a nominal home configuration

D. Metrics and Logging

A benchmarking node computes two primary metrics and logs all signals to CSV for offline analysis:

- **Joint-space tracking error:** Per-joint RMSE (radians) computed over the full trajectory between the physical and simulated joint positions.
- **End-to-end mirroring latency:** latency (ms) is estimated as the time offset between a physical joint-state sample and the corresponding simulated state update observed by the benchmark node.

To avoid cross-device clock misalignment, latency is computed using host-side reception timestamps when available (i.e., the time each message is received at the benchmarking node). All tests were performed in a static, calibrated

workspace to isolate synchronization behavior from external disturbances.

V. RESULTS AND DISCUSSION

The full experimental procedure was repeated ten times to produce a statistically meaningful dataset and to ensure that results were not affected by transient disturbances or intermittent network conditions. All ROS 2 logs were aggregated into unified datasets for synchronization analysis, temporal alignment, and error estimation. A separate measurement of internal ROS 2 message-handling latency was performed by timestamping messages at the output of the state acquisition node and at the input of the state replication node, within the same host machine. This measurement captures node-level processing and middleware handling, but excludes physical network transmission and simulator execution. Under these conditions, the subscription-to-publication transfer time within the ROS 2 pipeline averaged 17.38 ± 10.11 ms, indicating that internal ROS 2 message handling contributes a limited fraction of the total end-to-end latency observed in the Real-to-Sim pipeline.

Figure 6 presents the real and simulated joint trajectories for all six degrees of freedom over a 60-second interval, where the first 20 seconds correspond to a stationary phase and the 40-second trajectory execution begins at $t = 20$ s. Across all plots, the Digital Twin accurately replicates the temporal structure of the physical motion. The simulation closely follows the trajectory’s curvature, inflection points, and steady-state regions, demonstrating consistent propagation of joint commands through the mirroring pipeline.

For Joints 1–3 (Figures 6a to 6c), which correspond to joints with higher effective inertia, error accumulation is more noticeable during rapid accelerations. These deviations stem from combined effects of motor inertia, joint friction, and numerical integration differences between the physical controllers and Isaac Sim’s physics engine. In contrast, distal joints (Joints 4–6, Figures 6d to 6f) which carry less mass, exhibit lower deviations and more tightly synchronized behavior. This agrees with expected dynamic trends in articulated manipulators.

Distinct slope transitions, most clearly observed in Joint 3 and Joint 5 (Figures 6c and 6e), occur at approximately $t = 17$ s, 27 s, 37 s, and 47 s, corresponding directly to the four key poses defined in the trajectory. These inflection points confirm that the physical robot and the Digital Twin process identical waypoints and exhibit comparable phase changes, validating correct waypoint-to-trajectory mapping.

A stair-step pattern is consistently visible across all joint trajectories in both the real and simulated signals. This behavior results from discrete-time updates throughout the ROS 2 control pipeline, including fixed-rate joint-state publishing by the xArm driver, polling-based command application within the Isaac ROS 2 bridge articulation controllers, and the use of densely interpolated ROS 2 CLI commands that issue discretized position targets rather than a continuous command stream.

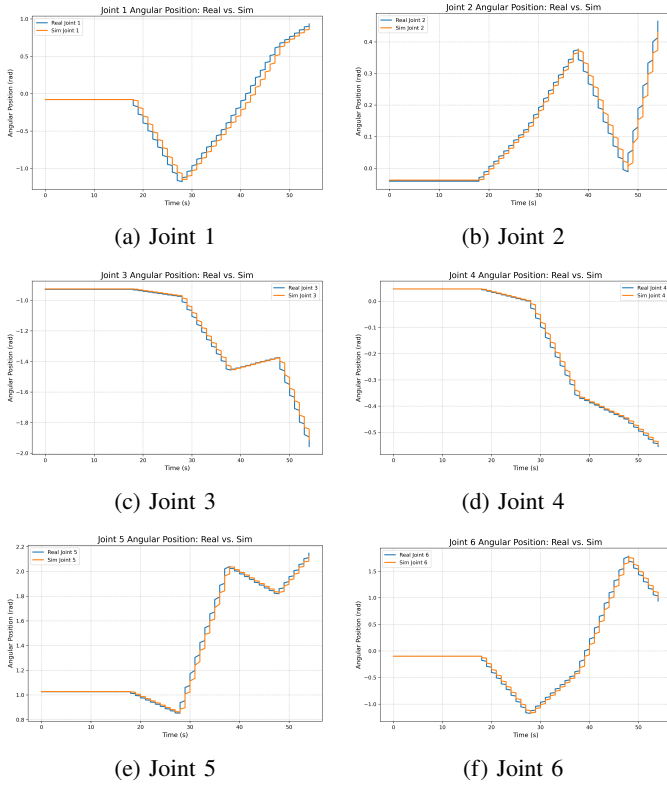


Figure 6: Real vs. Simulated Trajectory Tracking. Individual joint profiles showing the correlation between real and simulated angular states. Data is presented in radians to evaluate the synchronization accuracy of the Real2Sim framework.

Because both physical and simulated robots operate under the same discretized command sequence, this quantization appears symmetrically in both systems. Rather than an artifact of error, this symmetry indicates consistent replication of state updates.

Beyond visual inspection, Root Mean Square Error (RMSE) and Real-to-Sim delay were computed for every joint across all repetitions. Figure 7 consolidates these results by presenting the mean joint trajectories along with the delay histogram, providing a high-level view of synchronization quality.

Figure 7 summarizes RMSE and Real-to-Sim delay results across all trials. The delay distribution is centered around a mean of 362.95 ± 133.87 ms, which includes the internal ROS 2 latency previously measured. This value represents the total time between receiving a physical joint-state message and observing the corresponding simulated update. The distribution’s shape confirms that latency remains stable, without sporadic bursts or outliers that could compromise consistency during motion execution.

The joint-space tracking results show an average RMSE of 0.037 ± 0.007 rad ($\approx 2.11^\circ \pm 0.4^\circ$) across joints. These errors fall within acceptable tolerances for Digital Twin applications involving trajectory preview, workspace validation, offline planning, and task rehearsal. Most of the RMSE accumulated during high-acceleration segments, where simulation-

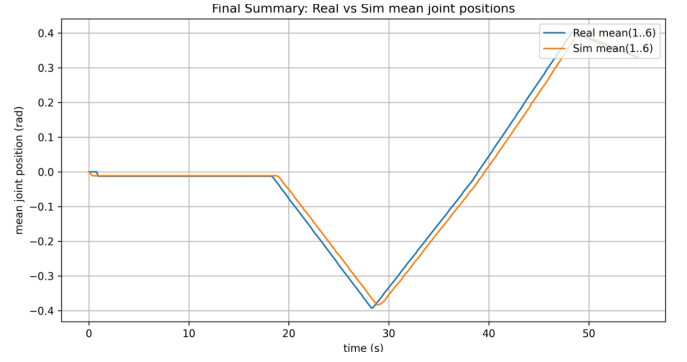


Figure 7: Average 6-DoF Trajectory Profile. Comparative time-series plot of the mean angular positions (rad), showing the global synchronization between the physical xArm6 and its Digital Twin

based numerical integration inherently differs from real motor dynamics.

Video demonstrations recorded during the experiments show qualitatively consistent motion between the physical system and its Digital Twin, without obvious phase mismatch under the tested trajectories. Gripper actuation and linear actuator replication behaved correctly throughout all trials, confirming the functional robustness of the xArm-Python-SDK integration and the associated ROS 2 service interfaces.

The achieved fidelity aligns with findings in related DT literature, where responsiveness depends heavily on communication middleware efficiency, message handling overhead, and GPU load in simulation-heavy environments. Some state-of-the-art implementations achieve lower latencies through predictive filtering or learned motion-extrapolation techniques, but such optimization methods were intentionally excluded in this work. The goal of the present system is not latency minimization but rather the demonstration of a reliable, modular, and extensible Isaac Sim-based Digital Twin architecture suitable for future cell-level research.

Taken together, these results establish a robust baseline for Real-to-Sim mirroring in Isaac Sim. The system’s modularity and high-fidelity rendering pipelines make it a strong foundation for future work involving dynamic scene reconstruction, synthetic data generation, human–robot collaboration, and immersive teleoperation.

VI. CONCLUSIONS AND FUTURE WORK

This paper presented a functional Digital Twin framework for a robotic cell built on NVIDIA Isaac Sim and a ROS 2 middleware layer. The system enables real-time state visualization, bidirectional communication, and reliable Real-to-Sim joint-state mirroring between physical and simulated manipulators. Through repeated trajectory experiments, we quantified end-to-end latency and joint-space tracking error, demonstrating that the DT provides stable and consistent mirroring suitable for trajectory testing, workspace evaluation, and offline experimentation.

Although the current implementation is not optimized for minimum latency, it establishes a research-oriented baseline and highlights practical factors, including bridge scheduling, simulator update rate, and calibration, that constrain strict real-time mirroring. The framework emphasizes modularity, ease of integration, and reproducibility, offering a foundation for extensions involving perception, dynamic scene updates, and advanced interaction modalities.

Future work will pursue five directions. First, integrating end-to-end 3D reconstruction will enable dynamic updates of the virtual environment via RGB-D scanning and continuous scene refinement, improving applicability to environment-aware tasks such as collision checking and workspace validation. Second, containerization will improve portability and reproducibility and facilitate deployment across multi-node or cloud-edge settings. Third, the framework will be extended toward closed-loop Sim-to-Real feedback by incorporating additional sensing and feedback signals to improve tracking stability under latency. Fourth, the system will be expanded toward immersive AR/VR teleoperation for intuitive interaction and remote supervision. Finally, the platform will be used to investigate learning-based policies (e.g., reinforcement learning) for assistance in human-robot collaboration, including safety-constrained behaviors.

In summary, this work establishes a robust, extensible baseline for developing Digital Twin systems in Isaac Sim and identifies key performance factors critical for future multi-robot, cell-level deployments. The presented architecture lays the groundwork for richer perception, more advanced interaction modalities, and higher-fidelity synchronization in next-generation industrial robotic Digital Twins.

REFERENCES

- [1] M. Achouch, M. Dimitrova, K. Ziane, S. Sattarpanah Karganroudi, R. Dhouib, H. Ibrahim, and M. Adda, "On predictive maintenance in industry 4.0: Overview, models, and challenges," *Applied Sciences*, vol. 12, no. 16, p. 8081, 2022.
- [2] R. Van Dinter, B. Tekinerdogan, and C. Catal, "Predictive maintenance using digital twins: A systematic literature review," *Information and Software Technology*, vol. 151, p. 107008, Jul 2022.
- [3] K. Y. H. Lim, P. Zheng, and C.-H. Chen, "A state-of-the-art survey of digital twin: Techniques, engineering product lifecycle management and business innovation perspectives," *Journal of Intelligent Manufacturing*, vol. 31, pp. 1313–1337, Nov 2019.
- [4] M. H. Zafar, E. F. Langás, and F. Sanfilippo, "Exploring the synergies between collaborative robotics, digital twins, augmentation, and industry 5.0 for smart manufacturing: A state-of-the-art review," *Robotics and Computer-Integrated Manufacturing*, vol. 89, p. 102769, Apr 2024.
- [5] Z. Huang, Y. Shen, J. Li, M. Fey, and C. Brecher, "A survey on ai-driven digital twins in industry 4.0: Smart manufacturing and advanced robotics," *Sensors*, vol. 21, p. 6340, Sep 2021.
- [6] M. Yazdi, "Digital twins and virtual prototyping for industrial systems," in *Reliability Engineering and Risk Analysis*, Springer Series in Reliability Engineering, pp. 155–168, Springer, 2024.
- [7] V. Kuts, N. Cherezova, M. Sarkans, and T. Otto, "Digital twin: Industrial robot kinematic model integration to the virtual reality environment," *Journal of Machine Engineering*, vol. 20, pp. 53–64, Jun 2020.
- [8] E. Bauer, A. Aburaia, H. Orsolits, and M. Aburaia, "Assessing unity's applicability for creating digital twins by creating a multi-robot digital twin," in *Proceedings of the 34th DAAAM International Symposium*, pp. 551–556, DAAAM International, 2023.
- [9] R. Ala-Laurinaho, J. Autiosalo, A. Nikander, J. Mattila, and K. Tammi, "Data link for the creation of digital twins," *IEEE Access*, vol. 8, pp. 228675–228684, 2020.
- [10] M. Singh, R. Srivastava, E. Fuenmayor, V. Kuts, Y. Qiao, N. Murray, and D. Devine, "Comparative study of digital twin developed in unity and gazebo," *Electronics*, vol. 14, p. 276, Jan 2025.
- [11] M. Singh, J. Kapukotuwa, E. L. S. Gouveia, E. Fuenmayor, Y. Qiao, N. Murry, and D. Devine, "Unity and ROS as a digital and communication layer for digital twin application: Case study of robotic arm in a smart manufacturing cell," *Sensors*, vol. 24, p. 5680, Aug 2024.
- [12] T. Luu, Q. Nguyen, T. Tran, M. Tran, S. Ding, J. Kua, and T. Hoang, "Enhancing real-time robot teleoperation with immersive virtual reality in industrial IoT networks," *The International Journal of Advanced Manufacturing Technology*, vol. 139, pp. 6233–6257, Aug 2025.
- [13] H. Yelchuri, D. K. Singh, N. K. Gnani, T. V. Prabhakar, and C. Singh, "Robotwin: A robotic teleoperation framework using digital twins," *arXiv preprint arXiv:2506.01027*, Jun 2025. Preprint.
- [14] D. P. Mtowe, L. Long, and D. M. Kim, "Low-latency edge-enabled digital twin system for multi-robot collision avoidance and remote control," *Sensors*, vol. 25, p. 4666, Jul 2025.
- [15] S. Nambiar, M. Jonsson, and M. Tarkian, "Automation in unstructured production environments using ISAAC SIM: A flexible framework for dynamic robot adaptability," *Procedia CIRP*, vol. 130, pp. 837–846, Jan 2024.
- [16] L. Tingelstad, "Digital twin of the kuka kmr iiwa: Developing a simulation framework using ros 2 and nvidia isaac sim," Master's thesis, Norwegian University of Science and Technology, 2023.
- [17] F. P. Audonnet, J. Grizou, A. Hamilton, and G. Aragon-Camarasa, "TELESIM: A modular and plug-and-play framework for robotic arm teleoperation using a digital twin," *arXiv preprint arXiv:2309.10579*, Sep 2023. Preprint.
- [18] Y. Karpichev, M. C. Zaouali, T. Charter, *et al.*, "A deployable and scalable ROS-Docker framework for multi-platform digital twin applications," *TechRxiv*, April 2025. Preprint.